

Unreal package

An **Unreal package** is a file that stores various kinds of data for Unreal Engine games, for example textures, sounds, map data and UnrealScript byte code. Many files types used by Unreal Engine games, such as .u, .utx, .uax, .umx, .unr, .ut2, .ut3 or .upk are Unreal Packages and internally have the same structure. Others like .int (localization file for international/English), .ucl (UT2004 cache library), .ka (Karma data), .ogg (OGG Vorbis music) or .dll (native code library) are not Unreal packages. One big difference between package files and non-package files is that network clients can download package files automatically if the server indicates that they are required, whereas non-package files cannot be downloaded this way.

Contents

- 1 Package file format
 - 1.1 Data types
 - 1.1.1 Compact index format
 - 1.1.2 Object references
 - 1.2 Package header
 - 1.3 Generation infos
 - 1.4 Heritage table
 - 1.5 Compressed chunks
 - 1.6 Name table
 - 1.7 Export table
 - 1.8 Import table
- 2 Links

Package file format

The package file format has evolved over time, but as long as the object types stored in the package file are known, older package format version are still readable for newer games. Obviously there's a limit to this compatibility. For example, UT's UnrealScript byte code format is different from that of e.g. UT2003 and there is no built-in backward compatibility for this difference. As a result, UT2003 may be able to load UT's .uax and .utx files, but not UT .u files. Also while there's no general restriction against loading newer package file versions, the version number was usually increased for good reasons.

Data types

Generally Unreal package files use little endian byte order. That means, a byte sequence 01 02 03 04 corresponds to the 4 byte value 0x04030201.

Commonly used data types are:

BYTE

An 8 bit numeric value, usually unsigned.

WORD

A 16 bit numeric value, often unsigned.

DWORD/INT

A 32 bit numeric value, either signed (INT) or unsigned (DWORD), sometimes used as bit flags field.

QWORD

A 64 bit numeric value, usually unsigned as bit flags field.

FLOAT

A 32 bit IEEE 754 single precision floating point value.

DOUBLE

A 64 bit IEEE 753 double precision floating point value.

INDEX



Upskill with Springboard+

Explore Courses in Tech, Business, and More – Many
Springboard+

UNICODEZ

A zero-terminated string of 16 bit characters.

STRING

An INDEX value (DWORD in UE3) as string length and type followed by either an ASCIIZ or UNICODEZ string. The length value's sign determines whether the string is stored using 8 bit (positive) or 16 bit (negative) characters. The length always refers to the number of characters, not the number of bytes, to follow and includes any trailing null character. Note that the empty string theoretically has three representations: length 0, length 1 and an ASCII null character, or length -1 and a Unicode null character. In practice the length 0 representation will be preferred.

Compact index format

The compact index format is a special way to store signed 32 bit values so they don't occupy as much space if their absolute value is relatively low. This format is used in many places in UE1 and UE2 packages, such as dynamic array lengths and name or object references, but became obsolete in UE3 where cooked package content can be compressed.

The rule of thumb for compact index values is: Lower absolute values can be stored with fewer bytes, the sign is stored separately. The sign and up to 6 value bits can be stored in the first byte, every additional byte stores up to 7 bits. That means, the values -63..63 are stored using a single byte, the values -8191..-64 and 64..8191 can be stored in two bytes, and so on.

The bits of the first byte:

7	6	5	4	3	2	1	0
Sign	More?	V5	V4	V3	V2	V1	V0

Vx are the absolute value's bits, "More?" is a flag signaling whether more bytes will follow and Sign is the value's sign bit. The second byte, if necessary, looks as follows:

7	6	5	4	3	2	1	0
More?	V12	V11	V10	V9	V8	V7	V6

The same notions as above apply. Values bits 13..19 are stored in byte 3, 20..26 in byte 4 and 27..30 in byte 5. Bit 31 would be the sign bit, which is already stored in byte 1. Data bits 4..7 in byte 5 are always zero. In practice, compact index values will usually occur in their 1 to 3 byte forms, rarely in the 4 byte form and probably never in their 5 byte form.

Object references

The import table, export table and many other places in a package file reference objects. Such references are stored as compact index value in UE1 and 2 and as DWORD value in UE3. Object references are resolved as follows:

- If the reference is zero, no object is referenced, i.e. NULL/None.
- If the reference is positive, the object must be looked up in the export table at index reference-1.
- If the reference is negative, the object must be looked up in the import table at index 1-reference.

Package header

Unreal package files always start with a header section that provides information about the content of the package. (Thanks to Eliot for the header fields introduced in UE3 (<http://eliotvu.com/page/unreal-package-file-format>).)

The package header is structured as follows:



- UT v346 writes version 69
- UT2004 v3369 writes version 128
- UT3 v2.1 writes version 512
- UE4 writes version -1 to -3, and is then followed by another two DWORDs(if negative!) in this particular order:
"UE3Version" (writes 868),
"UE4Version" (writes ~100 - ~300).

	WORD	Licensee version	Licensees may modify the package format. Their changes may render packages incompatible to standard packages, so they have this field as a separate way to handle their own change history.
			• UT writes version 0 • UT2003 writes version 28 • UT2004 writes version 29 • UT3 writes version 0
>= 249	DWORD	Header size	
>= 269	STRING	Folder name	
	DWORD	Package flags	
	DWORD	Name count	The number of entries in the package's name table.
	DWORD	Name offset	The byte position of the name table in the package file.
	DWORD	Export count	The number of entries in the package's export table.
	DWORD	Export offset	The byte position of the export table in the package file.
	DWORD	Import count	The number of entries in the package's import table.
	DWORD	Import offset	The byte position of the import table in the package file.
< 68	DWORD	Heritage count	The number of entries in the package's heritage table.
< 68	DWORD	Heritage offset	The byte position of the heritage table in the package file.
>= 415	DWORD	Depends offset	The byte position of the dependencies table in the package file. The number of entries is equal to Export count. (I may be wrong, could be Import count, but I am using Export count in UE Library so that has confused me, although it is commented out!).
>= 623	DWORD/DWORD/DWORD	ImportExportGuidsOffset, ImportGuidsCount, ExportGuidsCount	
>= 584	DWORD	ThumbnailTableOffset	

			below) for generations 1..GenerationCount, i.e. Generation Count values.
>= 245	DWORD	Engine version	(Optional: Engine licensee version)
>= 277	DWORD	Cooker version	(Optional: Cooker licensee version)
>= 334 (cooked)	DWORD	Compression flags	
>= 334 (cooked)	DWORD	Compressed chunks count	
>= 334 (cooked)	x * CompressedChunk	Compressed chunks list	List of compressed chunks (see Compressed chunks below) in the package.
>= ???	Array of Strings	An array of other cooked packages	Related to cooking.
>= ???	???	???	Related to textures.

Generation infos

Generation infos tell, how many export and name table entries previous versions of a package contained. This information is used for package map negotiation between the server and client in network games if they have different versions of a package.

Version	Type	Name	Description
	DWORD	Export count	The number of export table entries in that generation of the package.
	DWORD	Name count	The number of name table entries in that generation of the package.
>= 322	DWORD	Net object count	

Heritage table

Package versions less than 68 contain a heritage table instead of the generation infos. The number of entries and its size is specified in the package header.

The heritage table is nothing more but a list of GUID values of previous package generations. Any "post-heritage" engine that encounters a package with a heritage table will pick the last heritage entry as the package's GUID and assume the generation number 1.

Compressed chunks

Cooked Unreal Engine 3 packages can be compressed. The compressed chunks list maps the compressed parts of the file to uncompressed offset values. Every part of the package, except the header, can be part of a compressed chunk. The offsets specified in the package header actually refer to uncompressed positions.

Type	Name	Description
DWORD	Uncompressed offset	The start offset of this chunk in the completely uncompressed package.
DWORD	Uncompressed chunk size	The uncompressed size of this chunk.
DWORD	Compressed offset	The start offset of this chunk in the compressed package file.

the package header. Its entries are structured as follows:

Version	Type	Name	Description
	STRING	Name string	The string this name represents.
< 141	DWORD	Name flags	Flags associated with this name.
>= 141	QWORD	Name flags	Flags associated with this name.

Generally the name table does not contain any duplicate entries, including no different capitalizations. The only exception may be the name 'None', which may occur several times if the package was conformed to a previous version. Names in the name table are grouped by the generation they were added in. Every generation's name group starts with a new 'None' name. The indices of additional 'None' names correspond to the name counts of previous generations in the package header's generation info list.

Export table

The following things need to be done on this page or in this section:

- Fill in descriptions of unknown fields.

The export table specifies all objects the package contains. The size and position of the export table is specified in the package header. Its entries are structured as follows:

Version	Type	Name	Description
	INDEX	Class	The class of this object. None means this is a class itself.
	INDEX	Super	The object this object inherits from. Only used for struct, states, classes and functions.
	INT	Outer	The object containing this object. Resources may be contained in groups (which are subpackages), functions may be contained in states or classes, and so on.
>= 220	INDEX	Name	The index of this object's name in the name table.
< 195	INT	Archetype	The object's archetype reference.
	DWORD	Object flags	This object's flags.
>= 195	QWORD	Object flags	This object's flags.
	INDEX	Serial size	The size of this object's data in the package file. Some objects do not include any data.
(serial size > 0)	INDEX	Serial offset	The position of this object's data in the package file. This field is only present if the serial size is non-zero.
>= 220, < 543	INT	??? count	
>= 220, < 543	x * 12 bytes	???	
>= 247	DWORD	Export flags	
>= 322	INT	??? count	



export table entries are not strictly stored in any tree traversal order.

Like the name table, the export table may contain null objects if the package was conformed to an older version. Their indices correspond to the export counts of previous package generations.

Import table

The import table specifies objects from other packages that are required by objects in this package in some way. The size and position of the import table is specified in the package header. Its entries are structured as follows:

Type	Name	Description
INDEX	Package	The package to import this object from. Zero if the imported object is a package itself.
INDEX	Class	The class of the imported object.
INT	Outer	The object containing this imported object. Resources may be contained in groups (which are subpackages), functions may be contained in states or classes, and so on.
INDEX	Name	The index of the imported object's name in this package's name table.

Imported objects form trees with their package as root, but the import table entries are not strictly stored in any tree traversal order. Packages may import a package with the same name to reference native-only classes that don't have any UnrealScript parts. For example the Core package imports another package called Core containing classes such as Class or ClassProperty.

Links

- Unreal Package File Format (<http://eliotvu.com/page/unreal-package-file-format>) by Eliot van Uytfanghe
- UE Explorer (<http://eliotvu.com/portfolio/view/21/ue-explorer>) by Eliot van Uytfanghe - includes a .NET library for reading Unreal package files
- Unreal Tournament Package Tool (<http://www.acordero.org/projects/unreal-tournament-package-tool/>) by Antonio Cordero Balcázar - reads UE1 and UE2 packages
- UT Packages (<http://sourceforge.net/projects/utpackages/>) by Antonio Cordero Balcázar - Delphi library for reading UE1/2 packages
- UT Package File Format (<http://www.scribd.com/doc/54572848/UT-Package-File-Format>)    by Antonio Cordero Balcázar - File format documentation for UT and UT2004.
- UPK File Format (http://wiki.tesnexus.com/index.php/UPK_File_Format_-_XCOM:EU_2012) by the XCOM Community - File format explained by various sources.

Retrieved from "https://wiki.beyondunreal.com/index.php?title=Unreal_package&oldid=45963"